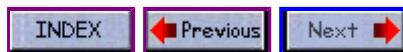


C Programming



ARRAYS

Little Boxes on the hillside

Arrays are a data structure which hold multiple variables of the same data type. Consider the case where a programmer needs to keep track of a number of people within an organisation. So far, our initial attempt will be to create a specific variable for each user. This might look like,

```
int name1 = 101;
int name2 = 232;
int name3 = 231;
```

It becomes increasingly more difficult to keep track of this as the number of variables increase. Arrays offer a solution to this problem.

An array is a multi-element box, a bit like a filing cabinet, and uses an indexing system to find each variable stored within it. In C, indexing starts at **zero**.

Arrays, like other variables in C, must be declared before they can be used.

The replacement of the above example using arrays looks like,

```
int names[4];
names[0] = 101;
names[1] = 232;
names[2] = 231;
names[3] = 0;
```

We created an array called *names*, which has space for four integer variables. You may also see that we stored 0 in the last space of the array. This is a common technique used by C programmers to signify the end of an array.

Arrays have the following syntax, using square brackets to access each indexed value (called an **element**).

$x[i]$

so that $x[5]$ refers to the sixth element in an array called x . In C, array elements start with 0. Assigning values to array elements is done by,

```
 $x[10] = g;$ 
```

and assigning array elements to a variable is done by,

```
g = x[10];
```

In the following example, a character based array named *word* is declared, and each element is assigned a character. The last element is filled with a zero value, to signify the end of the character string (in C, there is no string type, so character based arrays are used to hold strings). A `printf` statement is then used to print out all elements of the array.

```
/*  Introducing array's, 2  */
#include <stdio.h>

main()
{
    char word[20];

    word[0] = 'H';
    word[1] = 'e';
    word[2] = 'l';
    word[3] = 'l';
    word[4] = 'o';
    word[5] = 0;
    printf("The contents of word[] is -->%s\n", word );
}
```

Sample Program Output

The contents of word[] is Hello

©Copyright B Brown. 1984-1999. All rights reserved.

